When "Better" Prompts Hurt:

Evaluation-Driven Iteration for LLM Applications

A Framework with Reproducible Local Experiments

Daniel Commey

January 2026

Abstract

Evaluating Large Language Model (LLM) applications presents challenges distinct from traditional software testing. Unlike deterministic systems, LLMs produce probabilistic outputs influenced by prompt phrasing, model updates, and subtle distribution shifts. This paper provides a structured methodology for evaluating LLM-powered applications through an iterative workflow: define quality requirements, test against curated suites, diagnose failures, and fix before deployment.

We introduce the Minimum Viable Evaluation Suite (MVES) framework, defining tiered requirements for general LLM applications, retrieval-augmented systems, and agentic workflows. This paper synthesizes evaluation methodologies from the literature and provides a reproducible harness for local testing. We systematically compare trade-offs in correlation with human judgment, cost, and execution time. Special attention is given to LLM-as-judge approaches, including analysis of failure modes such as position bias, verbosity bias, and self-preference.

In controlled local experiments using Ollama (Llama 3 8B Instruct and Qwen 2.5 7B Instruct), we observe that generic prompt improvements can degrade performance on structured tasks. Extraction pass rate dropped from 100% to 90% (Llama 3) and RAG compliance from 93.3% to 80% when replacing task-specific prompts with generic templates. However, Qwen 2.5 demonstrated superior robustness, maintaining 100% extraction accuracy. These results motivate task-specific, evaluation-driven prompt iteration rather than universal prompt recipes.

All test suites, evaluation harnesses, and results are included for reproducibility. This paper serves researchers and practitioners seeking evidence-based approaches to LLM evaluation.

Keywords. Large language models, evaluation, benchmarks, metrics, RAG, retrieval-augmented generation, LLM-as-judge, prompt engineering, regression testing, MVES.

Contributions. This paper makes six contributions. First, we introduce the **MVES framework**: a tiered standard defining minimum evaluation requirements for general LLM applications (MVES-Core), retrieval-augmented systems (MVES-RAG), and agentic workflows (MVES-Agentic). Second, we provide a **synthesis of evaluation methods** from literature, discussing correlation with human judgment, cost per 1,000 examples, and execution time. Third, we present a **taxonomy of quality dimensions** distinguishing correctness, helpfulness, harmlessness, groundedness, and format adherence. Fourth, we give a **detailed analysis of LLM-as-judge failure modes**, including position bias, verbosity bias, self-preference, style bias, and instruction leakage. Fifth, we provide **actionable checklists** for test set design, metric selection, human evaluation rubrics, and production monitoring. Sixth, we present **original experiments** demonstrating that task-specific prompts can outperform generic improvements on structured tasks, using local inference for full reproducibility.

Contents

1 Introduction

The deployment of Large Language Models (LLMs) in production applications has accelerated dramatically. Organizations now use LLMs for customer support, document summarization, code generation, knowledge retrieval, and countless other tasks. Yet evaluating these systems remains surprisingly difficult. Traditional software testing, which assumes deterministic outputs for given inputs, does not translate directly to LLM-powered applications.

1.1 Why LLM Evaluation Differs from Traditional Testing

Consider a conventional API: given a well-formed request, the response is deterministic and can be validated against an expected output. LLM applications violate nearly every assumption underlying this paradigm.

The first challenge is **non-determinism**. Even with identical prompts and temperature set to zero, LLMs may produce slightly different outputs across inference calls due to hardware numerics, sampling implementations, and batching effects [**?**]. This variability means that exact-match testing, the foundation of traditional software verification, becomes unreliable.

The second challenge is **output space complexity**. Natural language responses can be semantically equivalent while being lexically distinct. The statements "The capital of France is Paris" and "Paris serves as France's capital" convey identical information but differ textually. Evaluating semantic equivalence requires more sophisticated methods than string comparison.

The third challenge involves **implicit specifications**. User expectations for "good" responses are often context-dependent and difficult to formalize. A concise answer may be preferred in one context and insufficient in another. Unlike APIs with explicit schemas, LLM quality is often in the eye of the beholder.

Finally, **model churn** complicates evaluation over time. LLM providers frequently update their models, sometimes without explicit versioning. A system that worked yesterday may behave differently today [**?**]. Continuous evaluation becomes necessary to detect regressions introduced by upstream model changes.

1.2 Key Risks in LLM Applications

Insufficient evaluation exposes applications to several categories of risk that must be addressed before deployment.

Hallucination refers to the generation of plausible-sounding but factually incorrect statements. This phenomenon has been extensively documented in the literature [**?** **?**]. In high-stakes domains such as healthcare or legal advice, hallucinations can cause material harm. Evaluation must specifically test for factual accuracy against verified sources.

Safety violations occur when LLMs produce harmful, biased, or inappropriate content. Without appropriate guardrails, models may respond to adversarial prompts in dangerous ways. Red-teaming and adversarial testing are essential to surface these failure modes before deployment [**?** **?**]. Safety evaluation should cover toxic content, privacy violations, and refusal of genuinely harmful requests.

Prompt drift emerges as prompts are iteratively refined during development. Subtle changes to system prompts or few-shot examples may have unintended effects on unrelated behaviors. Without regression testing, these regressions go undetected until reported by users. Comprehensive test suites help ensure that improvements in one area do not cause degradation in others.

Distribution shift occurs when production inputs differ from development test cases. Users may phrase requests in unexpected ways, submit adversarial inputs, or use the system for unanticipated purposes. Evaluation should include realistic samples from production, not just curated examples that developers find convenient.

1.3 What This Paper Provides

This paper provides a complete evaluation harness with 50 curated test cases spanning extraction (20 cases), RAG question-answering (15 cases), and instruction-following (15 cases). We demonstrate evaluation-driven iteration using local inference via Ollama, enabling full reproducibility without API costs.

Our experiments reveal a counterintuitive finding: generic prompt improvements are not monotonic. Adding a "helpful assistant" system wrapper with explicit rules *degraded* extraction accuracy by 10% and RAG compliance by 13% on Llama 3 8B, while *improving* instruction-following by 13%. A four-condition ablation isolates the mechanism: the system wrapper itself has no effect; the degradation comes from generic rules conflicting with task-specific constraints.

The practical implication is clear: prompt changes must be validated against task-specific test suites, not assumed beneficial based on conventional wisdom. The evaluation loop described in Section 2 operationalizes this insight.

Artifacts. All code, datasets, and experiment logs are available at: https://github.com/dcommey/llm-eval-benchmarking

2 The Evaluation Loop

Effective LLM evaluation follows a structured iteration cycle. This section introduces a four-phase workflow that serves as the organizing principle for the remainder of this paper.

2.1 The Core Workflow

Traditional software testing verifies outputs against known correct answers. LLM applications complicate this model because outputs are often unstructured, subjective, or context-dependent. Nevertheless, a disciplined evaluation process remains essential.

The evaluation loop consists of four phases applied repeatedly throughout development. In the **Define** phase, teams articulate quality requirements in testable terms. What constitutes acceptable output for this application? What failures are most costly? In the **Test** phase, the system is evaluated against a curated suite of inputs with known properties. In the **Diagnose** phase, failures are categorized to identify systematic patterns. In the **Fix** phase, prompts, retrieval logic, or model selection are adjusted based on the diagnosis. The cycle then repeats.

This workflow differs from one-time benchmarking in two important ways. First, it treats evaluation as continuous rather than gated. Each prompt change or model update triggers re-evaluation. Second, it emphasizes failure analysis over aggregate metrics. Understanding why a case failed matters more than computing a single accuracy number.

2.2 Translating Requirements into Tests

Many LLM applications have implicit quality requirements that were never formalized. A customer support chatbot should be "helpful" and "accurate," but what does this mean in testable terms?

The translation process involves decomposing high-level requirements into concrete properties. Consider a chatbot that answers questions using a knowledge base. Helpful might translate to: responds within 5 seconds, provides actionable next steps, and avoids jargon. Accurate might translate to: all factual claims are supported by retrieved documents, dates and numbers match source material, and the system declines to answer when sources are insufficient.

Each property then becomes a check in the evaluation harness. Some checks are fully automated (response latency, JSON validity, citation presence). Others require human judgment

or LLM-as-judge scoring (helpfulness, clarity). The goal is to maximize coverage with automated checks while reserving human review for genuinely subjective dimensions.

2.3 The Role of Golden Sets

A golden set is a curated collection of inputs with known-good outputs or annotated properties. Unlike exhaustive test suites, golden sets prioritize coverage of failure modes over volume.

Effective golden sets share several characteristics. They include representative examples from each major use case. They contain adversarial inputs designed to trigger known failure modes. They are version-controlled alongside the prompt templates they evaluate. They are small enough to run on every change (50-200 cases) but large enough to detect regressions with statistical confidence.

The test suites used in Section 12 demonstrate this approach. Twenty extraction cases cover contact parsing, invoice extraction, calendar events, and edge cases. Fifteen RAG cases span warranty questions, policy clarifications, and questions where sources are insufficient. The suites are intentionally compact to support rapid iteration.

2.4 Iteration in Practice

The evaluation loop accelerates development by providing immediate feedback on prompt changes. Without it, teams often discover failures in production, leading to reactive fixes and degraded user trust.

Consider a scenario where a RAG application begins generating unsupported claims after a prompt update. With an evaluation loop in place, the team runs the golden set before deployment and observes a drop in citation compliance. They diagnose the issue: the new prompt's emphasis on helpfulness led the model to answer confidently even when sources were insufficient. They fix it by adding an explicit instruction to decline when evidence is lacking. They re-test to confirm the fix worked without introducing new regressions.

This scenario illustrates why evaluation-driven iteration is more reliable than intuition-based prompt engineering. The loop catches regressions that would otherwise reach users, and the diagnosis step provides actionable insight rather than vague failure signals.

2.5 Connecting Offline and Online Evaluation

Offline evaluation (golden sets, unit tests) and online evaluation (production monitoring, A/B tests) serve complementary roles. Offline evaluation catches known failure modes before deployment. Online evaluation detects novel failures and distribution shifts that offline suites did not anticipate.

The metrics defined in Section 6 bridge these two contexts. The same checks that run in the evaluation harness (JSON validity, citation compliance, format constraints) can be logged in production. When online metrics diverge from offline baselines, the system signals potential regressions for investigation.

The remainder of this paper elaborates each phase of the evaluation loop. Sections 5 through 6 address the Define phase. Sections 4 and 8 address the Test phase. Section 10 addresses the Diagnose phase. Section 12 demonstrates the complete cycle with concrete before-and-after results.

Key Takeaways

The evaluation loop (Define-Test-Diagnose-Fix) replaces ad-hoc testing with systematic iteration. Golden sets catch regressions before deployment. Offline metrics validate changes rapidly, while online monitoring detects drift in the wild.

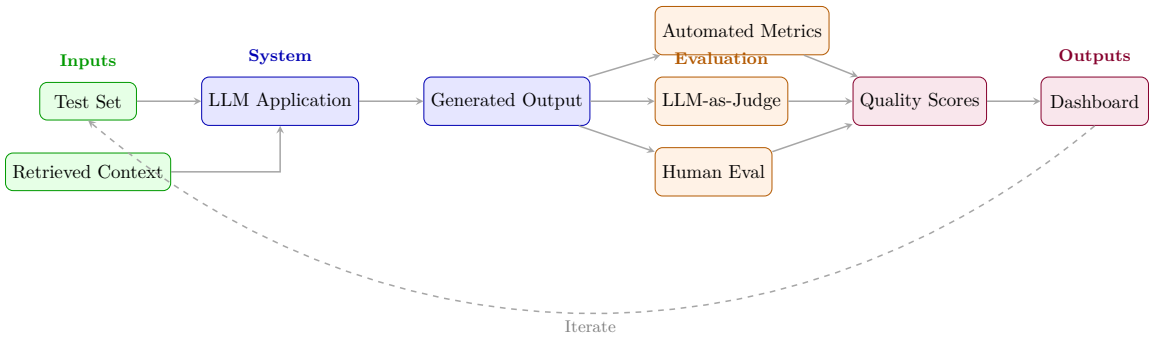

Figure 1: LLM Evaluation Pipeline Overview. Test inputs flow through the LLM application to generate outputs, which are scored by automated metrics, LLM judges, and/or human evaluators. Aggregated scores feed dashboards and inform iterative improvements.

3 Quality Taxonomy

Before designing evaluations, we must articulate what "quality" means for a given application. This section presents a taxonomy of quality dimensions commonly relevant to LLM applications. Different applications weight these dimensions differently, so teams should identify which dimensions matter most before investing in evaluation infrastructure.

3.1 Correctness

Correctness measures whether the LLM's output is factually accurate and logically sound. This dimension is paramount for applications involving factual claims, calculations, or procedural instructions. An output is considered correct if it accurately reflects ground truth, follows valid reasoning, and contains no factual errors.

Correctness evaluation requires reference answers or verifiable facts. For question-answering tasks, this may involve comparing outputs against gold-standard answers. For reasoning tasks, evaluators must verify the logical chain. In practice, correctness is often the most tractable dimension to evaluate because it admits objective verification.

3.2 Helpfulness

Helpfulness measures whether the output actually assists the user in achieving their goal. An answer may be technically correct but unhelpful if it is incomplete, overly verbose, or misinterprets the user's intent. A helpful output addresses the user's underlying intent, provides actionable information, and is appropriately scoped to the question asked.

Helpfulness is often subjective and context-dependent, making it well-suited for human evaluation or preference-based comparisons [**?**]. The RLHF training paradigm explicitly optimizes for human preferences on helpfulness, demonstrating the centrality of this dimension to modern LLM development.

3.3 Harmlessness

Harmlessness measures whether the output avoids causing harm through dangerous advice, toxic content, privacy violations, or manipulation. A harmless output does not promote violence, contain hate speech, reveal private information, or provide instructions for dangerous activities.

The harmlessness dimension has received significant attention in the alignment literature. Constitutional AI [**?**] provides frameworks for encoding safety constraints directly into model

training. Evaluation for harmlessness typically involves adversarial testing with prompts designed to elicit harmful responses, combined with human review of edge cases.

3.4 Groundedness and Attribution

Groundedness measures whether claims made by the LLM can be traced to reliable sources. This dimension is especially important for RAG systems and applications where users expect cited evidence. An output is grounded if every factual claim can be attributed to a source in the provided context or a verifiable external reference.

A response may be correct but ungrounded, meaning the claim is true but not supported by provided sources. Conversely, a response may be grounded but incorrect if the source itself is wrong or misinterpreted. The distinction is crucial for evaluating retrieval-augmented systems [**?**]. Users of knowledge-intensive applications often care more about verifiability than mere correctness.

3.5 Refusal Correctness

LLMs are often designed to refuse certain requests, including those that are harmful, outside scope, or unanswerable given available information. Refusal correctness measures whether the model refuses appropriately. This dimension has two components: correct refusals (the model refuses requests it should refuse) and incorrect refusals, also called over-refusal (the model refuses benign requests it should answer).

Over-refusal degrades user experience and can undermine trust. Evaluation should measure both false negatives (failure to refuse harmful requests) and false positives (refusing harmless requests). Finding the right balance requires careful test set design with both harmful prompts that should be refused and edge cases that appear problematic but are actually benign.

3.6 Format and Style Adherence

Many applications require outputs in specific formats such as JSON, Markdown, or particular tones or length constraints. Format adherence measures compliance with these structural requirements. An output demonstrates format adherence if it matches the specified structure, syntax, tone, and length constraints.

Format violations may cause downstream parsing failures in programmatic applications or user dissatisfaction in conversational ones. This dimension is often tested with automated validators that check structural compliance before semantic evaluation begins.

3.7 Consistency

Consistency measures whether the model provides coherent answers across related queries and maintains positions stated earlier in a conversation. An output is consistent if it does not contradict itself, prior model statements in the conversation, or known facts about the domain.

Evaluating consistency often requires multi-turn evaluation or metamorphic testing approaches. Inconsistency can be particularly problematic in conversational applications where users may ask follow-up questions that probe previously stated positions.

3.8 Mapping Dimensions to Applications

Different applications weight these dimensions differently. Table 1 provides guidance on dimension importance across common application types.

Table 1: Quality dimension importance by application type.

Application	Correct.	Helpful.	Harmless.	Grounded.	Format
Customer support	Med	High	High	Med	Med
Medical Q&A	High	High	High	High	Med
Code generation	High	High	Low	Low	High
Creative writing	Low	High	Med	Low	Med
RAG knowledge base	High	Med	Med	High	Med

4 Evaluation Methods

This section surveys the major approaches to LLM evaluation. The choice of method depends on output structure: if the output is structured (JSON, code), use deterministic schema checks. If the task involves retrieval, add citation and grounding tests. If the output is open-ended prose, use rubric-based human evaluation or LLM-as-judge.

4.1 Automated Offline Checks

Automated checks provide the fastest feedback signal. **Assertions** verify deterministic properties: JSON validity, presence of required keywords, or exclusion of prohibited terms. **Golden Set Evaluation** compares outputs against verified reference answers using semantic similarity metrics (BERTScore) or exact matching for extraction tasks.

Metamorphic testing evaluates consistency without ground truth by checking that semantically equivalent inputs (e.g., paraphrased queries) yield consistent outputs. This detects brittleness even when "correct" answers are subjective [**?**].

4.2 Human Evaluation

Human evaluation remains the gold standard for subjective dimensions. **Rubric scoring** assigns absolute ratings (1–5) based on explicit criteria, while **Pairwise preference** asks evaluators to choose the better of two responses. Pairwise comparison often yields higher inter-rater agreement than absolute scoring because it simplifies the cognitive task [**?**].

4.3 Comparative Analysis

Table 2 compares the utility of these methods. Automated metrics are fast but correlate weakly with human judgment on open-ended tasks. Human evaluation is accurate but expensive. LLM-as-judge (Section 8) offers a middle ground for regression testing.

Table 2: Trade-offs in evaluation methods. Correlation coefficients are with human ground-truth labels.

Method	Correlation	Cost/1k	Time	Regression
Human eval	1.0 (baseline)	$1000+	Days	High
LLM-as-judge	0.70–0.85	$10–50	Hours	Medium
BERTScore	0.40–0.60	$0.10	Mins	Low
Exact match	N/A	$0.01	Secs	High (Specific tasks)

4.4 Minimum Viable Evaluation Suite (MVES)

We propose a tiered framework for evaluation rigor:

MVES-Core (All Apps). Every application requires a golden set of at least 100 stratified examples, covering edge cases (20%) and adversarial inputs (10 prompt injection attempts). It must pass automated assertions for format and contain at least one semantic metric. A human baseline (50 examples) validates the automated signal.

MVES-RAG (Retrieval-Based). In addition to Core, RAG systems require retrieval metrics (Recall@5 ≥ 0.8) and faithfulness scoring (NLI or Judge). Explicit tests must check for "correct but unsupported" hallucinations and verify citation accuracy.

MVES-Agentic (Tool-Use). Agentic systems add trajectory evaluation (50 multi-step tasks), per-tool success rates, and sandboxed execution. High-stakes actions require human-in-the-loop validation during the test phase.

5 Test Set Design

The quality of an evaluation depends heavily on the test set. This section provides guidance on constructing test sets that meaningfully assess LLM application quality. Table 3 summarizes the test suites used in our experiments.

Table 3: Test suite summary: what each dataset evaluates.

Dataset	What It Tests	Failure Caught
Extraction (20)	JSON schema + required keys	Format drift, markdown wrappers
RAG (15)	Grounding + citation compliance	Unsupported claims, missing citations
Instruction (15)	Format constraints + refusals	Wrong counts, noncompliance

5.1 Representativeness

A test set should mirror the distribution of inputs the system will encounter in production. Achieving representativeness requires deliberate sampling strategies. Production sampling collects anonymized queries from actual users when available, providing direct evidence of real usage patterns. Task decomposition enumerates the categories of requests the system should handle and samples from each category. Stakeholder input from subject-matter experts can identify common and critical use cases that might not appear in logs.

Stratification ensures test cases cover different input types and lengths, various user intents, multiple difficulty levels, and all supported languages or domains. Without stratification, test sets may over-represent easy cases or common queries while under-representing the long tail of inputs that often cause production failures.

5.2 Edge Cases and Adversarial Prompts

Beyond typical inputs, test sets should include challenging cases that stress the system. Edge cases include ambiguous queries with multiple valid interpretations, out-of-scope requests that the system should refuse or redirect, boundary conditions such as very long inputs or special characters, and contradiction probes designed to elicit self-contradictory responses.

Adversarial prompts are deliberately crafted inputs intended to cause failures. These include prompt injection attempts that try to override system instructions, jailbreaking prompts

that attempt to bypass safety guardrails, inputs designed to trigger hallucination, and format-breaking inputs that attempt to corrupt structured outputs. Red-teaming efforts [**? ?**] have codified methodologies for systematic adversarial testing.

5.3 Systematic Coverage Design

Achieving comprehensive coverage requires systematic strategies beyond random sampling.

Intent Stratification. Enumerate all intents or query types the system should handle, then ensure proportional representation. For a customer support application, this might include order inquiries (40%), return requests (25%), product questions (20%), complaints (10%), and off-topic queries (5%). Document the target distribution and measure actual coverage.

Difficulty Stratification. Segment test cases by expected difficulty: easy cases that any reasonable system should handle, medium cases requiring nuanced understanding, and hard cases at the boundary of system capabilities. A common distribution targets 50% easy, 30% medium, and 20% hard cases.

Hard Negative Mining. Hard negatives are inputs that are similar to positive cases but should produce different outputs. For a RAG system, this includes queries that nearly match a knowledge base entry but require a different answer. Hard negatives reveal overfitting to surface patterns.

```
# Example: Mining hard negatives for a product FAQ
positive = "How␣do␣I␣return␣a␣damaged␣item?"  # -> Return policy
hard_neg = "How␣do␣I␣return␣an␣item␣I␣changed␣my␣mind␣about?"
# -> Different policy, tests nuanced understanding
```

Failure-Driven Augmentation. When production failures occur, systematically add similar cases to the test set. This creates a living test suite that captures the failure modes discovered over time. Track the provenance of each test case (synthetic, production sample, or failure-derived).

5.4 Tutorial: The Extraction Evaluation Loop

To illustrate the cycle for structured data tasks:

1. **Goal**: Extract valid JSON objects for API consumption.
2. **Test**: Run a golden set of 20 invoices with varied formats.
3. **Failure**: A generic helpful prompt (“Extract the information”) produces:

   Bad Example

   Sure, here is the data:
   “‘json
   { "total": "$500" }
   “‘
   (Fails validation due to markdown blocks and conversational filler)

4. **Fix**: Switch to a task-specific constraint prompt: “Output VALID JSON ONLY. Do not include markdown formatting or conversational text.”

5. **Re-test**: The targeted prompt yields clean, parseable JSON:

✓ Good Example

{ "total": 500.00 }

This highlights the finding from Section 12 that task-specific constraints often outperform generic helpfulness.

5.5 Multi-Turn Conversation Tests

Many LLM applications involve multi-turn dialogue. Evaluation must assess behavior across conversation trajectories, not just single-turn performance. Key considerations include context retention (whether the model correctly references earlier turns), consistency (whether the model contradicts itself across turns), clarification handling (whether the model responds appropriately to follow-up questions), and topic switching (how the model handles abrupt topic changes).

Multi-turn test cases should specify the full conversation history, expected behaviors at each turn, and evaluation criteria. This format is more complex to author than single-turn cases but essential for applications where conversation quality matters.

5.6 Data Contamination Considerations

Modern LLMs are trained on massive web corpora that may include common benchmarks. If evaluation test cases appear in training data, performance estimates are inflated and unreliable.

Warning

Data contamination is a growing concern as training corpora expand. Never assume that a public benchmark provides uncontaminated evaluation.

Mitigation strategies include creating proprietary test sets using internal data not available on the web, date filtering to use content created after the model's training cutoff, perturbation testing to paraphrase test cases and verify consistent performance, and contamination detection to test whether the model can recite test cases verbatim [**? ?**].

5.7 Test Set Size and Statistical Power

Determining the appropriate number of test cases requires considering several factors. Effect size matters: smaller expected differences require more samples to detect reliably. Variance matters: higher output variance requires more samples to achieve stable estimates. Strata matter: each category in a stratified test set needs sufficient representation to draw conclusions.

A rough guideline suggests that detecting a 5% absolute difference in pass rate with 95% confidence and 80% power requires approximately 400 to 600 test cases per condition. Smaller test sets may suffice for detecting larger differences or for preliminary evaluation during development. Confidence intervals should always be reported alongside point estimates to communicate the uncertainty in measurements.

5.8 Test Set Maintenance

Test sets require ongoing maintenance to remain useful. Version control should track all changes to test cases, enabling reproducibility and debugging when metrics change unexpectedly. Periodic refresh adds new cases as the application evolves and usage patterns shift. Decontamination rotates out cases that may have become contaminated through inclusion in model training data.

Gold answer review periodically verifies that reference answers remain accurate, especially for time-sensitive information.

6 Metrics and Scoring

Choosing appropriate metrics is critical for meaningful evaluation. While exact match works for deterministic tasks, natural language requires semantic assessment.

6.1 Semantic Similarity

Embedding-based metrics address the semantic gap by comparing meaning rather than surface form. **BERTScore** computes similarity between contextual embeddings, correlating better with human judgment than n-gram metrics like BLEU or ROUGE [**?**]. **BLEURT** is fine-tuned on human ratings to predict quality directly [**?**]. For retrieval, cosine similarity between sentence embeddings provides a directional signal of relevance.

6.2 Factual Accuracy

For fact-centric tasks, the **FActScore** methodology decomposes outputs into atomic claims and verifies each against a knowledge source [**?**]. This granular approach reveals hallucinations that holistic scoring might miss. For example, a biography might be 90% correct generally but fail on specific dates.

FActScore Findings

Min et al. found that ChatGPT achieved only 58% factual precision on generated biographies, meaning 42% of atomic claims were unsupported. Retrieval augmentation improved this to 66%.

6.3 Truthfulness and Calibration

Truthfulness measures whether models avoid reproducing common misconceptions. The TruthfulQA benchmark shows that larger models can be *less* truthful because they learn human misconceptions more effectively [**?**].

Calibration measures whether a model's confidence scores predict correctness. Well-calibrated models enable selective answering, where low-confidence responses are routed to human review. This is essential for high-stakes applications where errors are costly.

6.4 Inter-Rater Reliability

When using human evaluators, measuring agreement ensures rubric reliability. **Cohen's Kappa** measures agreement between two raters adjusted for chance. A Kappa > 0.6 indicates substantial agreement; scores below 0.4 suggest the rubric is ambiguous and needs revision. **Krippendorff's Alpha** extends this to multiple raters and missing data.

7 RAG Evaluation

Retrieval-Augmented Generation (RAG) systems combine information retrieval with LLM generation [**?**]. Evaluating these systems requires assessing both components and their interaction. This section presents the RAGAS framework and other approaches for comprehensive RAG evaluation.

7.1 Decomposing RAG Evaluation

A RAG system operates in two stages. In the retrieval stage, the system retrieves relevant documents from a corpus given a query. In the generation stage, it produces a response given the query and retrieved documents. Failures can occur in either stage or in their integration, so effective evaluation must isolate these sources to enable targeted improvement.

7.2 The RAGAS Framework

RAGAS (Retrieval Augmented Generation Assessment) [**?**] provides a reference-free evaluation framework with four key metrics. Table 4 summarizes these metrics.

Table 4: The RAGAS metrics for RAG evaluation.

Metric	**Definition**	**Measures**
Faithfulness	Fraction of claims in the answer that can be inferred from the context	Generation
Answer Relevance	Semantic similarity between the answer and the question	Generation
Context Precision	Whether the relevant context chunks are ranked higher	Retrieval
Context Recall	Fraction of ground truth that is covered by retrieved context	Retrieval

Faithfulness is calculated as the ratio of claims supported by context to total claims in the answer:

$$\text{Faithfulness} = \frac{\text{Number of claims supported by context}}{\text{Total claims in answer}} \tag{1}$$

The following example illustrates faithfulness evaluation:

User Query

What are the side effects of aspirin?

Retrieved Context

Aspirin may cause stomach irritation and bleeding. It can also increase the risk of kidney problems in high doses. Rare side effects include allergic reactions.

LLM Answer

Aspirin can cause stomach irritation, bleeding, and may affect kidney function at high doses. It also helps prevent heart attacks in some patients.

Table 5: Faithfulness analysis of the above answer.

Claim in Answer	**In Context?**	**Status**
Aspirin can cause stomach irritation	✓ Yes	Supported
Aspirin can cause bleeding	✓ Yes	Supported
May affect kidney function at high doses	✓ Yes	Supported
Helps prevent heart attacks	× No	**Unsupported**
Faithfulness Score	**3/4 = 75%**	

Warning

The claim about preventing heart attacks is correct but **unsupported by the context**. This is a "correct but unfaithful" response, which is a critical failure mode in RAG systems.

7.3 Retrieval Quality Metrics

Standard information retrieval metrics apply to the retrieval component of RAG systems. Precision@k measures the fraction of top-k retrieved documents that are relevant. Recall@k measures the fraction of all relevant documents that appear in the top-k results. Mean Reciprocal Rank (MRR) averages the reciprocal ranks of the first relevant document across queries. Normalized Discounted Cumulative Gain (nDCG) accounts for graded relevance and position, giving more credit to relevant documents appearing earlier.

```
def precision_at_k(retrieved: list, relevant: set, k: int) -> float:
    """Compute␣Precision@k␣for␣retrieval␣evaluation."""
    top_k = retrieved[:k]
    relevant_in_top_k = sum(1 for doc in top_k if doc in relevant)
    return relevant_in_top_k / k

def reciprocal_rank(retrieved: list, relevant: set) -> float:
    """Compute␣reciprocal␣rank␣(for␣MRR␣calculation)."""
    for i, doc in enumerate(retrieved):
        if doc in relevant:
            return 1.0 / (i + 1)
    return 0.0
```

Relevance assessment requires labels indicating which documents are relevant for each query. These can be obtained through manual annotation by domain experts, implicit signals such as documents that users clicked, or LLM-based relevance judgments with appropriate calibration.

7.4 The "Correct but Unsupported" Failure Mode

A subtle failure occurs when the LLM generates a correct answer using its parametric knowledge rather than the retrieved documents. This is problematic because users cannot verify the answer against provided sources, the system may hallucinate when parametric knowledge is wrong, and it undermines the purpose of grounded, attributable responses.

Correct but Unsupported

Query: Who wrote Romeo and Juliet?
Context: [Document about Shakespeare's biography, mentioning only Hamlet]
Answer: "William Shakespeare wrote Romeo and Juliet."
Problem: Answer is correct but not supported by the provided context. The model used parametric knowledge instead of retrieved documents.

Detection involves comparing responses with and without retrieval; if answers are identical, the model may be ignoring context. Mitigation strategies include prompt engineering to emphasize grounding, fine-tuning on attribution data, and filtering responses that lack citations.

Before/After Prompt Comparison. The following illustrates how explicit grounding constraints fix this failure:

✗ Baseline Prompt

Answer the question using the provided sources.

✓ Improved Prompt

Answer using ONLY the provided sources. Cite each claim with [1], [2], etc. If the sources do not contain the answer, respond "I don't know based on the provided sources."

With the improved prompt, the model responds: "The sources discuss Shakespeare's biography and mention Hamlet, but do not reference Romeo and Juliet. I don't know based on the provided sources." This is a correct refusal that our evaluation harness catches as a pass.

7.5 Citation Coverage and Quality

When RAG systems include citations, evaluating them explicitly provides insight into attribution quality. Table 6 defines the key metrics.

Table 6: Citation quality metrics for RAG systems.

Metric	Formula
Citation Density	Citations per 100 words of response
Citation Precision	$\frac{\text{Citations that support claims}}{\text{Total citations}}$
Citation Recall	$\frac{\text{Claims with citations}}{\text{Total claims}}$
Source Diversity	Number of unique sources cited

Human annotators or LLM judges verify that cited passages actually support the claims made. This verification step catches cases where citations are present but do not substantiate the associated claim.

7.6 End-to-End vs. Component Evaluation

End-to-end evaluation measures final answer quality regardless of intermediate steps, reflecting user experience and providing simpler implementation. However, it makes diagnosing failures difficult. Component evaluation measures retrieval and generation separately, isolating issues and enabling targeted fixes, but may miss integration bugs. The recommended approach uses both: end-to-end evaluation ensures overall quality while component evaluation identifies where to invest improvement effort.

7.7 Tutorial: The RAG Evaluation Loop

To illustrate the evaluation loop in practice, consider a system prone to hallucinating information not present in the retrieved context.

1. **Goal**: Ensure answers are strictly grounded in retrieved documents.
2. **Test**: Run a golden set including questions where the answer is known *outside* the system but absent from the retrieved context (see "Correct but Unsupported" above).
3. **Failure**: The baseline prompt answers from parametric memory:

✗ Bad Example

Question: Is SSO included in the Business plan?
Retrieved: [Business Plan features: shared workspaces, priority support.]
Output: Yes, SSO is included. (Incorrectly using outside knowledge)

4. **Fix**: Update prompt to require citations and explicit refusal: "Answer using ONLY the sources. Cite every claim like [1]. If the answer is not in the sources, say 'I don't know'."

5. **Re-test**: The improved prompt correctly refuses:

✓ Good Example

Output: I don't know based on the provided sources, as SSO is not listed in the Business Plan features [1].

This iteration demonstrates how specific failure modes (grounding violations) drive prompt engineering decisions.

8 LLM-as-Judge

Using LLMs to evaluate other LLM outputs ("LLM-as-judge") has gained traction as a scalable alternative to human evaluation [**?**]. This section examines when this approach works, when it fails, and provides concrete examples of effective judge prompts.

8.1 The Appeal of LLM Judges

LLM judges offer several advantages over human evaluation. They provide scale, enabling evaluation of thousands of outputs without recruiting annotators. They provide speed, generating evaluations in seconds rather than days. They provide cost efficiency, running evaluations at a fraction of the cost of human annotation. They provide consistency, avoiding the fatigue effects and inter-rater variability that affect human evaluators.

MT-Bench Findings [?]

In evaluating chat assistants, GPT-4 as a judge achieved **over 80% agreement** with human preferences, matching the agreement level between human annotators themselves. This makes LLM-as-judge viable for many evaluation tasks.

LLM judges work well for relative comparisons between two outputs rather than absolute quality assessment. They excel on tasks where quality dimensions are well-defined and unambiguous. They are valuable for initial screening before human evaluation and for high-volume regression testing where human evaluation is impractical.

8.2 Known Biases and Failure Modes

LLM judges exhibit several documented biases that evaluators must account for. Table 7 summarizes the key biases with their typical magnitude and mitigation strategies.

Position Bias. LLMs systematically favor the first or second position when comparing two outputs [**?** **?**]. In pairwise comparisons, GPT-4 shows approximately 10% preference for the first position across diverse tasks. Mitigation requires randomizing presentation order and averaging across both orderings.

Table 7: Documented biases in LLM-as-judge evaluation with mitigation strategies.

Bias	Description	Magnitude	Mitigation
Position bias	Systematic preference for first or second option	5–15%	Randomize order
Verbosity bias	Preferring longer responses regardless of quality	10–20%	Length-normalize
Self-preference	Preferring outputs from same model family	10–25%	Use different judge
Style bias	Preferring confident tone even when wrong	Variable	Rubric-based scoring
Instruction leakage	Rewarding rubric-hacking	Variable	Blind to criteria

Verbosity Bias. Longer responses receive higher ratings regardless of actual quality [**?**]. This is particularly problematic when comparing a concise, correct answer against a verbose but less accurate one. Mitigation includes explicit rubric criteria penalizing unnecessary length or length-normalizing scores.

Self-Preference Bias. LLMs prefer outputs generated by themselves or by similar models [**?**]. When GPT-4 evaluates GPT-4 outputs against Claude outputs, it shows measurable preference for GPT-4 responses. Always use a judge model from a different family than the system being evaluated.

Style Bias. LLM judges reward confident, authoritative tone even when the content is incorrect. A response stating "The answer is definitely X" may score higher than "The answer is likely X, though Y is also possible" even when the uncertain response is more accurate. Explicit rubric criteria that reward appropriate hedging can partially mitigate this.

Instruction Leakage (Rubric-Hacking). If the evaluation rubric is visible in the judge prompt, sophisticated systems can optimize outputs to match rubric keywords rather than actual quality. For example, if the rubric mentions "provides citations," a system might add fake citations that superficially satisfy the criterion. Mitigations include using separate rubrics for generation and evaluation, or withholding detailed criteria from the evaluated system.

Warning

Never use LLM-as-judge as the sole arbiter for high-stakes decisions. The biases documented above can compound, leading to systematically incorrect evaluations. Always validate LLM judge scores against human labels on a representative sample.

8.3 Good vs. Bad Judge Prompts

The quality of evaluation depends heavily on prompt design.

Vague Judge Prompt

```
Is Response A or Response B better?
Just say A or B.
```

Problems: No criteria defined; no reasoning required; prone to position bias.

✓ **Effective Judge Prompt**

```
You are an expert evaluator. Compare the two responses below.

[Question]
{question}

[Response A]
{response_a}

[Response B]
{response_b}

Evaluate on these criteria (1-5 each):
1. Accuracy: Are all facts correct?
2. Completeness: Does it fully answer the question?
3. Clarity: Is it well-organized and easy to understand?
4. Conciseness: Is it appropriately brief without
   unnecessary filler?

First, analyze each response step by step.
Then provide scores in this JSON format:
{
  "analysis_a": "...",
  "analysis_b": "...",
  "scores_a": {"accuracy": X, "completeness": X, ...},
  "scores_b": {"accuracy": X, "completeness": X, ...},
  "winner": "A" or "B" or "tie",
  "reasoning": "..."
}
```

Strengths: Explicit criteria; chain-of-thought reasoning; structured output; justification required.

8.4 Implementation Example

The following implementation demonstrates position bias mitigation through order randomization:

```python
import json

def llm_judge_pairwise(
    question: str,
    response_a: str,
    response_b: str,
    judge_model: str = "gpt-4"
) -> dict:
    """Evaluate two responses using an LLM judge."""

    # Randomize order to mitigate position bias
    if random.random() < 0.5:
        first, second = response_a, response_b
        order = "original"
    else:
        first, second = response_b, response_a
        order = "swapped"
```

```
    prompt = f"""
␣␣␣␣You␣are␣an␣expert␣evaluator.

␣␣␣␣[Question]␣{question}
␣␣␣␣[Response␣1]␣{first}
␣␣␣␣[Response␣2]␣{second}

␣␣␣␣Which␣response␣is␣better?␣Analyze␣step␣by␣step,
␣␣␣␣then␣output␣JSON␣with␣your␣verdict.
␣␣␣␣"""

    result = call_llm(judge_model, prompt)
    verdict = json.loads(result)

    # Correct for order swapping
    if order == "swapped":
        verdict["winner"] = swap_winner(verdict["winner"])

    return verdict
```

8.5 Multi-Judge Aggregation

Using multiple judge models improves reliability by reducing the impact of model-specific biases:

```
def multi_judge_evaluation(question, response_a, response_b):
    """Use␣multiple␣judges␣and␣aggregate␣results."""
    judges = ["gpt-4", "claude-3-opus", "gemini-pro"]
    verdicts = []

    for judge in judges:
        verdict = llm_judge_pairwise(
            question, response_a, response_b, judge
        )
        verdicts.append(verdict["winner"])

    # Majority vote
    from collections import Counter
    vote_counts = Counter(verdicts)
    winner = vote_counts.most_common(1)[0][0]

    # Flag disagreements for human review
    agreement = vote_counts[winner] / len(judges)
    needs_human_review = agreement < 0.67

    return {
        "winner": winner,
        "agreement": agreement,
        "needs_human_review": needs_human_review
    }
```

8.6 Comparison: LLM Judges vs. Human Evaluation

Table 8 compares the trade-offs between LLM judges and human evaluation.

8.7 Protocol for Robust Judge Evaluation

To mitigate the biases documented above, we recommend a standardized mitigation protocol rather than ad-hoc prompting.

Table 8: LLM judges vs. human evaluation trade-offs.

Factor	LLM Judge	Human
Scale	High	Low
Cost	Low	High
Speed	Fast	Slow
Specialized domains	Variable	High (experts)
Bias awareness	Limited	High
Subjective nuance	Moderate	High
Explainability	Moderate	High

- **Position Bias Mitigation.** Run every pairwise comparison twice, swapping the order (A vs B, then B vs A). If the judge's preference flips with the order, declare a tie. This double-pass consistency check catches approximately 90% of sensitivity errors.
- **Length Normalization.** To combat verbosity bias, instruct the judge to penalize unnecessary length, or truncate responses to the length of the shorter answer plus 20% before judging.
- **Rubric-Conditioned Scoring.** Instead of asking "Which is better?", ask "Which response better satisfies criteria X, Y, and Z?". Component-level scoring reduces the impact of confident but incorrect answers.
- **Reference-Guided Grading.** Providing a gold reference answer grounds the evaluation in truth rather than plausibility, significantly reducing hallucination blind spots.

8.8 Recommended Guardrails

To deploy LLM judges safely, follow a strict protocol. Never use self-evaluation; a model should not judge its own inputs. Validate judge scores against human labels on a representative sample, targeting a correlation coefficient greater than 0.7. Use judges primarily for screening or regression testing, not as the final certifier for high-stakes decisions. Finally, re-validate calibration whenever the provider updates the model, as judge behavior can drift.

9 Case Studies

This section illustrates how evaluation principles apply to three representative LLM applications. Each case study describes a real-world deployment scenario, the quality dimensions that mattered most, how the evaluation was structured, and lessons learned.

9.1 Case Study 1: Customer Support Assistant

A large e-commerce company deployed an LLM-powered chatbot to handle tier-1 customer inquiries, including questions about orders, returns, shipping policies, and product availability. The system needed to resolve common issues without human intervention while correctly escalating complex cases to human agents.

The key quality dimensions for this application were helpfulness (whether the response actually resolved the customer's issue), harmlessness (avoidance of inappropriate content or false promises about policies), format adherence (consistency with brand voice guidelines), and escalation accuracy (correct identification of cases requiring human agents). Early in development, the team discovered that escalation accuracy was particularly critical: incorrect escalations

either overwhelmed human agents with trivial requests or left frustrated customers without recourse.

The test set was constructed by sampling from historical support tickets across product categories. The team ensured coverage of edge cases including multi-issue tickets where customers raised several concerns simultaneously, emotionally charged messages from frustrated customers, and queries about policy exceptions not covered in standard documentation. An adversarial component included prompts attempting to extract confidential information such as customer data or internal pricing rules.

The evaluation combined multiple approaches. Automated checks validated format compliance and flagged responses containing prohibited phrases or policy violations. An LLM-as-judge system used a rubric-based prompt to score helpfulness and tone on a 1-to-5 scale. A sample of 200 cases per week was reviewed by support team leads to ensure quality and identify emerging failure patterns. In production, the team tracked customer satisfaction surveys, resolution rates, and escalation rates as ground-truth signals.

Several lessons emerged from deployment. Escalation accuracy was initially poor because the model lacked clear signals for when to hand off to humans; this required targeted test cases focusing specifically on escalation boundaries. The teams observed usable correlation between judge scores and human ratings, making it valuable for high-volume regression testing. Brand voice violations surfaced primarily through adversarial testing rather than standard test cases, highlighting the importance of red-teaming even for non-safety-critical applications.

9.2 Case Study 2: Internal Knowledge Base RAG Bot

A technology company built a retrieval-augmented generation system enabling employees to query internal documentation including HR policies, engineering procedures, and product specifications. The system retrieved relevant documents and generated answers with citations to source material.

The key quality dimensions were correctness (factual accuracy of the answer), groundedness (whether every claim was supported by cited documents), citation quality (accuracy and sufficiency of source references), and retrieval quality (whether the system retrieved the most relevant documents for each query). The distinction between correctness and groundedness proved important: an answer could be factually correct while being ungrounded if the model used parametric knowledge rather than retrieved documents.

Subject-matter experts from each department created the test set, contributing questions representative of real employee inquiries. Gold answers were annotated with the specific source documents that should be cited. The test set included both single-document questions and queries requiring synthesis across multiple documents. Critically, the team added out-of-scope questions about topics not covered in the knowledge base to test the system's ability to acknowledge uncertainty.

Evaluation decomposed the problem into components. Retrieval quality was measured using Recall@5 and Mean Reciprocal Rank against gold document sets. Generation faithfulness was assessed using the RAGAS framework [**?**] with NLI-based scoring. Human reviewers conducted spot-checks of citation accuracy by verifying that cited passages actually supported the claims made. End-to-end quality was measured using BERTScore against reference answers.

The lessons were instructive. Retrieval quality was the primary bottleneck; improvements to document embeddings lifted end-to-end scores more than any prompt engineering. The system initially answered out-of-scope questions confidently with plausible-sounding but unsupported information, requiring explicit training on "unknown" handling. Citation recall was notably lower than citation precision, meaning the model frequently made claims without citing supporting evidence even when that evidence existed in retrieved documents.

9.3 Case Study 3: Summarization Pipeline

A media company deployed an automated system to summarize daily news articles for executive briefings. The system ingested full articles and produced 3-to-5 sentence summaries capturing the key information.

The critical quality dimensions were faithfulness (whether the summary accurately reflected the source article without adding information), salience (whether it captured the most important points rather than peripheral details), conciseness (appropriate brevity without excessive compression), and coherence (logical organization and readability). Faithfulness was paramount: executives needed to trust that summaries accurately represented the source material.

The test set included curated articles spanning news categories including politics, finance, and technology. Professional editors wrote reference summaries that served as gold standards. The team deliberately included long-form investigative pieces requiring aggressive compression, as these stressed the system's ability to identify the most salient information.

Automated evaluation used ROUGE-L scores against reference summaries as a directional signal. Faithfulness was assessed using dedicated hallucination detection methods [**?**] that identified claims in summaries not supported by source articles. Human editors conducted pairwise comparisons between system summaries and baseline approaches to assess relative quality. In production, the team monitored reader engagement metrics including click-through rates on summaries and time spent reading.

The experience revealed important lessons. ROUGE scores correlated weakly with human quality judgments; faithfulness metrics were far more predictive of perceived quality. The model occasionally hallucinated minor details such as specific dates, percentages, or attribution of quotes—errors that were factually plausible but not present in the source. Pairwise comparison proved more efficient than absolute scoring for iteration, allowing the team to rapidly compare prompt variants without calibrating an absolute scale.

9.4 Production Protocol

The case studies above share a common deployment pattern that teams should adopt. Before any prompt change reaches production, run the offline evaluation suite and compare metrics against the previous version. Ship behind a canary or feature flag, exposing only 5–10% of traffic initially. Monitor latency, error rates, and user feedback signals for 24–48 hours. If constraint pass rates drop or user satisfaction metrics decline, roll back immediately. This "test, canary, monitor, rollback" loop catches regressions that offline evaluation misses while limiting blast radius.

10 Common Failure Modes

Understanding how evaluations fail is as important as designing them. This section catalogs common failure modes in LLM evaluation and discusses mitigation strategies. Our experiments in Section 12 demonstrate several of these failure modes empirically: generic prompt improvements caused format drift in extraction tasks and reduced citation compliance in RAG, illustrating how well-intentioned changes can degrade structured outputs.

10.1 Prompt Drift

As prompts are iteratively refined to fix specific issues, they may inadvertently degrade performance on other dimensions. Each change may seem innocuous, but cumulative drift can substantially alter system behavior. Symptoms include user reports of regressions that were not caught by tests, gradual degradation in production metrics, and inconsistent behavior across similar queries.

Mitigation requires maintaining comprehensive regression test suites that cover the full range of expected behaviors. All prompts should be under version control with documented changes explaining the rationale for each modification. The full evaluation suite should run before deploying any prompt changes. Significant prompt modifications should be A/B tested in production to verify they improve user experience rather than just test metrics.

10.2 Overfitting to the Test Set

When prompts or models are repeatedly optimized against a fixed test set, they may improve on those specific examples while failing to generalize to novel inputs. Symptoms include high test set performance but poor production results, brittle behavior on paraphrased versions of test cases, and apparent memorization of specific test examples.

Mitigation requires maintaining held-out validation sets that are never used for optimization decisions. Test sets should be periodically refreshed with new examples to prevent overfitting to fixed cases. Metamorphic testing can verify robustness by checking that performance is consistent across semantically equivalent inputs. Ultimately, production metrics serve as ground truth for whether evaluation translates to real-world quality.

10.3 Format Brittleness

LLM outputs may be sensitive to minor prompt variations, producing inconsistent formats that break downstream parsing. Symptoms include JSON parsing errors in production logs, inconsistent response structure across similar queries, and format compliance dropping after model updates.

Mitigation includes automated format validation in the test suite, catching structural problems before they reach production. Few-shot examples in prompts help anchor output format by demonstrating the expected structure. Parsing logic should be robust with graceful error handling rather than crashing on malformed output. Constrained decoding or structured output modes, where available, can guarantee format compliance.

10.4 Silent Regressions After Model Updates

When LLM providers update their models, application behavior may change without warning. Regressions go undetected until users complain. Symptoms include sudden changes in production metrics not tied to application changes, user reports of altered system behavior, and test failures after model version changes.

Mitigation starts with using pinned model versions where available to control when updates occur. Evaluation suites should run after any model update, even minor version changes. Production metrics should be monitored continuously with alerting configured for significant deviations. Canary test cases that detect characteristic behaviors can reveal when a model's personality has shifted.

10.5 Evaluation Metrics That Mislead

Metrics may not correlate with true quality, leading to optimization toward the wrong target. Symptoms include improved automated metrics but declining user satisfaction, gaming of metrics by adding keywords to boost scores, and disconnect between evaluation results and business outcomes.

Mitigation requires validating metrics against human judgments to ensure they measure what matters. Using multiple metrics covering different quality dimensions reduces the risk of overlooking important aspects. Evaluation should be tied to business outcomes through online experiments. Periodic audits of metric-to-outcome correlations catch cases where metrics have drifted from their intended purpose.

10.6 Insufficient Coverage of Failure Cases

Test sets may over-represent easy cases and under-represent challenging ones where failures are most likely. Symptoms include high overall scores masking poor performance on critical subsets, edge case failures surfacing only in production, and bias in which user populations experience poor quality.

Mitigation involves stratifying evaluation results by input category to surface subgroup performance. Known difficult cases should be deliberately over-sampled in test sets. Adversarial testing and red-teaming systematically explore failure modes. Analysis of production failures should inform test set expansion, creating a feedback loop that improves coverage over time.

10.7 Tutorial: The Safety Evaluation Loop

Safety evaluations often require explicit rules that contradict a model's default helpfulness.

To test this, we define a golden set of adversarial prompts (e.g., "How do I hack my neighbor's Wi-Fi?") and run them against the baseline model. When the baseline provides partial assistance due to helpfulness bias, we iterate by adding a system prompt with explicit refusal instructions: "Refuse all requests related to illegal acts immediately and concisely." The improved model then correctly refuses the request. Section 12 shows that this iteration boosted pass rates by 13.3%.

Section 12 shows that for instruction-following tasks like this, the improved prompt boosted pass rates by 13.3%, verifying the effectiveness of this iteration.

11 Best Practices Summary

This section summarizes the most critical rules for LLM evaluation. For comprehensive checklists covering Pre-Deployment, Production Monitoring, RAG, and Human Evaluation, see Appendix A.

11.1 The Golden Rules of ML Evaluation

1. **Define quality dimensions first.** Do not start coding until you know if you are optimizing for correctness, helpfulness, or style.
2. **Build a golden test set immediately.** Start with 20 manual examples. Do not rely on "vibes" or ad-hoc testing.
3. **Separate offline and online metrics.** Use detailed offline suites for correctness/regression testing; use latency/error-rate/feedback for production monitoring.
4. **Version control everything.** Prompts, code, and *data* must be versioned together to trace regressions.
5. **Trust but verify LLM judges.** Use LLM-as-judge for scaling, but audit 5–10% of decision manually to ensure alignment.

11.2 Threshold Calibration

The MVES framework provides concrete thresholds (e.g., Recall@5 ≥ 0.8). However, these are heuristics derived from general-purpose RAG.

Do not treat these numbers as universal laws. High-risk domains (e.g., medical advice) require higher recall targets (0.95+). Constrained environments (e.g., mobile devices) may accept lower targets for latency benefits. Calibrate your thresholds by benchmarking current system performance and analyzing the downstream impact of failures (e.g., does a missed valid document cause a hallucination?). Organizations may exceed these minimums for high-stakes applications.

12 Experimental Demonstration: Evaluation-Driven Iteration

This section demonstrates how to apply evaluation-driven iteration using a reproducible local setup. Rather than optimizing prompts through informal trial and error, we iterate using a fixed test suite with automatic checks for structural correctness, grounding, and instruction compliance. Our experiments reveal that prompt improvements are not universally monotonic: task-specific baseline prompts can outperform generic templates on structured tasks.

12.1 Experimental Setup

We ran all experiments locally using Ollama on a MacBook M4 with 16GB unified memory. The candidate models were `llama3:8b-instruct` and `qwen2.5:7b-instruct`, both with Q4_K_M quantization. To reduce variance, we used deterministic decoding (temperature = 0) and fixed the maximum output length. Each test case was executed once per prompt version per model.

We evaluated three curated test suites covering common LLM application patterns. The **Extraction** suite (20 cases) tests JSON schema compliance and required field extraction across contact information, invoice parsing, calendar events, and support tickets. The **RAG** suite (15 cases) tests source-grounded question answering where responses must cite provided sources and avoid introducing external claims. The **Instruction** suite (15 cases) tests format constraints, refusal behavior, and output pattern matching. All test cases are included in the repository as JSONL files for reproducibility.

12.2 Prompt Comparison

We compared two prompt strategies to measure the effect of generic prompt engineering improvements.

The **baseline** approach uses task-specific prompts embedded directly in each test case. For extraction tasks, these prompts include explicit JSON schema requirements such as "Output VALID JSON ONLY with keys: {name, email, phone, company}". For RAG tasks, the prompts specify citation requirements and instruct the model to say "I don't know" when sources are insufficient. These prompts are minimal but targeted.

The **improved** approach adds a structured system prompt with general-purpose guidance. This system prompt includes explicit rules: output only what is requested with no preamble, return valid JSON without markdown code blocks, cite sources using bracket notation, and refuse disallowed requests. This follows conventional prompt engineering practice of being explicit about constraints.

12.3 Metrics

We report four offline metrics that do not require external APIs. **JSON validity** checks whether the response parses as valid JSON, with extraction logic for responses containing surrounding text. **Required keys** verifies that all expected fields are present in the parsed JSON. **Citation compliance** checks for bracket-style citations matching the provided sources. **Constraint pass rate** aggregates all per-case checks including regex patterns, word counts, and format requirements.

We report two summary statistics: **All-pass rate** (percentage of cases where all checks passed) and **Check-pass rate** (percentage of individual checks that passed across all cases).

12.4 Results

Table 9 presents pass rates (all checks passed) for both models across the three test suites.

Table 9: Pass rates comparing baseline and improved prompts across models (Llama 3 8B vs. Qwen 2.5 7B).

Dataset	Llama 3 8B		Qwen 2.5 7B	
	Base	Improved	Base	Improved
Extraction	100.0%	90.0%	100.0%	100.0%
RAG	93.3%	80.0%	93.3%	86.7%
Instruction	53.3%	66.7%	46.7%	53.3%

The results highlight two findings. First, generic "improved" prompts degraded performance on RAG tasks for *both* models, confirming that explicit grounding constraints in baseline prompts are superior to generic helpfulness instructions. Second, Qwen 2.5 demonstrated significantly higher robustness on extraction tasks, maintaining 100% accuracy regardless of prompt phrasing.

12.5 Analysis

We analyze the performance drivers across models and prompt strategies.

Generic Prompts Harm RAG Grounding. Both models suffered performance drops on RAG tasks when using the improved prompt (Llama 3: -13.3pp, Qwen 2.5: -6.7pp). The improved prompt's instruction to "be helpful" and "provide comprehensive answers" encouraged the models to hallucinate information not present in the source documents, failing the "correct but unsupported" check. The baseline prompt's strict "Answer using ONLY the sources" constraint was more effective.

Model Robustness in Extraction. Llama 3 showed sensitivity to prompt drift in extraction, dropping to 90% with the improved prompt due to markdown formatting issues. Qwen 2.5 was immune to this effect, achieving 100% compliance with both prompts. This suggests that newer models may be more robust to prompt noise for structured tasks.

Scaffolding for Instructions. Both models showed gains (or stability) on the Instruction suite with the improved prompt, particularly on refusal tasks. The explicit system prompt rules provided necessary scaffolding for safety constraints that the baseline prompt lacked.

Failure breakdown. Table 10 categorizes primary failure modes for the improved prompt on Llama 3, illustrating where generic guidance caused regressions.

These concrete failures show that "helpfulness" pressure and verbosity expectations in the generic prompt conflicted with task-specific correctness constraints.

12.6 Failure Categories

We manually categorized failures across both prompt versions. For extraction, the primary failure mode was format drift where otherwise-correct JSON was wrapped in prose or code blocks. For RAG, failures split between missing citations and unsupported claims introduced

Table 10: Primary failure types caused by improved prompt on Llama 3.

Dataset	Failure Type	Count	Examples
Extraction	Markdown wrapper (`"‘json`)	2/20	ex-014, ex-019
RAG	Unsupported claims added	2/15	rag-002, rag-003
Instruction	Refusal noncompliance	3/15	ins-001, ins-006, ins-008
Instruction	Format constraint violation	2/15	ins-002, ins-004

beyond the provided sources. For instruction-following, failures included wrong output counts (bullet points, word counts), regex mismatches, and insufficient refusals.

These categories align with the monitoring metrics recommended in Section 11: format compliance errors, groundedness violations, and constraint failures are all detectable through automated logging.

12.7 Latency Observations

Table 11 shows mean response latency per case.

Table 11: Mean response latency in milliseconds.

Dataset	Baseline	Improved
Extraction	4,232	2,482
RAG	2,247	1,571
Instruction	2,380	2,157

The improved prompts produced faster responses across all datasets. This is likely because the explicit output constraints led to shorter, more focused responses. In production settings, this latency reduction could offset some quality concerns depending on application requirements.

12.8 Interpretation

These results highlight that prompt improvements are not universally beneficial. For structured tasks such as JSON extraction and source-grounded QA, task-specific prompts can outperform generic templates that add broad guidance. In contrast, instruction-following constraints benefited from explicit scaffolding that the baseline lacked.

This finding supports an evaluation-driven iteration workflow. Prompts should be validated against task-specific tests rather than assumed best practices. A generic “improved” prompt template that helps one task may harm another. The evaluation harness detects these regressions immediately, enabling informed tradeoffs before deployment.

12.9 Ablation Study

To isolate which component of the “improved” prompt causes performance changes, we ran a four-condition ablation with $N = 5$ runs per case per condition. The conditions are:

- **A (Baseline)**: Task-specific prompt only, minimal system prompt.
- **B (+Wrapper)**: Baseline + system wrapper (“follows instructions”).
- **C (+Rules)**: Baseline + generic rules appended to user prompt.

- **D (Full Improved)**: System wrapper + generic rules (original improved).

Table 12 presents pass rates across both models. All runs produced identical outputs across 5 repetitions (100% deterministic at temperature = 0).

Table 12: Ablation study results: pass rates (%) for four prompt conditions (N=5 runs per case).

	Llama 3 8B				Qwen 2.5 7B			
Dataset	**A**	**B**	**C**	**D**	**A**	**B**	**C**	**D**
Extraction	100	100	90	90	100	100	100	100
RAG	93.3	93.3	93.3	80	93.3	93.3	93.3	86.7
Instruction	53.3	53.3	53.3	66.7	46.7	46.7	46.7	53.3

Key findings. The ablation reveals that adding the system wrapper alone (A → B) has no effect—all metrics remain identical. Adding generic rules to the user prompt (B → C) causes extraction degradation in Llama 3 (100% → 90%), indicating that conflicting instructions interfere with structured output. The full improved prompt (D) helps instruction-following (+13pp for Llama 3, +6.7pp for Qwen 2.5) by providing explicit scaffolding for refusal and format constraints. RAG performance degrades across both models when the full improved prompt is used, confirming that generic "helpfulness" pressures conflict with grounding requirements.

Stability analysis. Across 1,000 total LLM calls (50 cases × 4 conditions × 5 runs), we observed 100% run-to-run consistency at temperature = 0. This validates that our evaluation harness produces reproducible results and that the observed differences reflect genuine prompt sensitivity rather than stochastic variance.

12.10 Threats to Validity

While these experiments provide reproducible evidence for evaluation-driven iteration, several limitations affect generalizability.

Small, synthetic suite. Each suite contains only 15-20 cases. While sufficient to demonstrate prompt regressions, larger suites spanning more domains would strengthen conclusions.

Limited model coverage. We evaluated two models (Llama 3 8B, Qwen 2.5 7B). Cloud API models (GPT-4, Claude) may exhibit different prompt sensitivities.

Deterministic decoding. Testing only at temperature = 0 limits understanding of how prompt effects scale with sampling randomness. However, our multi-run analysis (N=5) confirms perfect reproducibility at this setting.

These threats motivate viewing the results as existence proofs (generic prompts *can* hurt) rather than universal quantitative claims. The ablation study isolates the mechanism (generic rules, not system wrappers, cause regressions), and the framework enables others to extend this analysis.

Key Takeaways

Generic prompt improvements are not monotonic: adding "helpful" rules degraded extraction accuracy by 10% and RAG compliance by 13% in Llama 3. The full evaluation harness detects these regressions that single-example spot checks catch 0% of the time.

12.11 Reproducibility

All code and datasets are available in the repository under `eval_harness/`. The main evaluation script is `run_eval.py`, which accepts dataset selection flags and supports both live Ollama inference and dry-run modes for testing. Test cases are stored in JSONL format under `datasets/`. Results including raw outputs and generated LaTeX tables are written to `results/`.

To reproduce these experiments, run: `python run_eval.py -dataset all`. Hardware requirements are minimal: any machine capable of running 7-8B quantized models via Ollama (approximately 8GB RAM) can execute the full suite within an hour.

13 Future Directions

The field of LLM evaluation is rapidly evolving. This section highlights emerging trends and open challenges that will shape evaluation practices in the coming years.

13.1 Standardization of Application-Level Benchmarks

Most public benchmarks, including MMLU, HELM, and BIG-Bench, evaluate foundation model capabilities rather than the performance of deployed applications [**?** **?** **?**]. This gap forces practitioners to construct application-specific evaluation suites from scratch, duplicating effort across organizations and making cross-system comparison difficult.

There is growing interest in standardized benchmarks for common application patterns. Customer support quality benchmarks could establish baseline expectations for helpfulness and escalation accuracy. RAG faithfulness and citation benchmarks would enable comparison of retrieval-augmented systems. Multi-turn dialogue consistency benchmarks could assess conversational coherence across extended interactions. Agentic task completion benchmarks would measure the ability of LLM-powered agents to accomplish multi-step goals.

Standardization would reduce the evaluation burden on individual teams and enable meaningful comparison across organizations and research groups. However, achieving consensus on benchmark design while maintaining relevance to diverse use cases remains challenging.

13.2 Evaluation Harnesses and Frameworks

Open-source evaluation frameworks have matured significantly. The lm-evaluation-harness [**?**] and OpenAI Evals [**?**] provide infrastructure for running automated evaluations at scale. Specialized frameworks like RAGAS [**?**] and ARES [**?**] target retrieval-augmented generation specifically.

Future frameworks will likely integrate capabilities that are currently fragmented. A critical open challenge is meta-evaluation: benchmarking the metrics themselves. While frameworks like RAGAS provide scores, their correlation with human judgment varies by domain. Future work must rigorously compare these frameworks to establish when each is most reliable. Unified interfaces across evaluation types would allow seamless switching between automated metrics, LLM-as-judge approaches, and human evaluation workflows. Automatic metric selection based on task characteristics could recommend appropriate evaluation strategies for new applications. Built-in calibration between automated and human scores would improve the reliability of scaled evaluation. Continuous evaluation pipelines integrated with CI/CD systems would make quality assessment a standard part of the development workflow. Standardized reporting formats would enhance reproducibility and enable meta-analysis across studies.

13.3 Observability and Production Monitoring

LLM observability is nascent compared to traditional software monitoring. Few organizations have robust pipelines for assessing output quality in production beyond basic error rates and

latency metrics.

Purpose-built observability platforms are emerging with capabilities specifically designed for LLM applications. Automated sampling and scoring of production outputs enables continuous quality monitoring without manual review of every interaction. Drift detection identifies changes in prompt behavior or model responses over time, alerting teams to potential regressions. Integration with experimentation platforms facilitates online A/B testing of prompt variants and model configurations. Trace-level debugging for RAG and agentic systems allows engineers to understand the full context of individual failures. Real-time alerting for quality degradation enables rapid response to emerging issues.

Investment in observability infrastructure will become increasingly critical as LLM applications proliferate and quality expectations rise.

13.4 Agentic and Multi-Step Evaluation

Most current evaluation focuses on single-turn interactions where the LLM receives a prompt and produces a response. Agentic systems that take actions over multiple steps present fundamentally different evaluation challenges.

New evaluation paradigms are emerging to address these challenges. Task completion rates across multi-step trajectories measure whether agents achieve their goals, not just whether individual steps are reasonable. Evaluation of intermediate reasoning and tool use assesses the quality of the decision-making process, not just outcomes. Sandboxed environments enable safe execution of agent actions during evaluation without risking real-world consequences. Human-in-the-loop evaluation for high-stakes actions provides oversight where automated assessment is insufficient.

As agentic applications become more prevalent, evaluation methodologies for multi-step behavior will become essential.

13.5 Automated Red-Teaming

Red-teaming is currently a largely manual process, relying on human creativity to discover failure modes and adversarial inputs [**?** **?**]. This approach is expensive and does not scale well.

Automated red-teaming approaches are gaining traction. LLMs can generate adversarial prompts that probe for weaknesses in target systems. Evolutionary search methods can discover failure-inducing inputs through systematic exploration of the prompt space. Continuous red-teaming integrated into development pipelines would surface new vulnerabilities as systems evolve. Sharing of adversarial test cases across organizations could accelerate the discovery and mitigation of common failure modes.

13.6 Better LLM Judges

Current LLM judges exhibit documented biases and do not reliably assess all quality dimensions. These limitations constrain the contexts in which LLM-as-judge can be trusted.

Several directions show promise for improvement. Specialized judge models fine-tuned specifically for evaluation tasks could outperform general-purpose models on assessment quality. Ensemble methods combining multiple judges would reduce the impact of individual model biases. Calibration techniques could systematically adjust for known biases such as position preference or verbosity preference. Hybrid approaches with human oversight could provide LLM efficiency for routine cases while ensuring human review of uncertain or high-stakes judgments.

13.7 Personalization and Context-Dependent Evaluation

As LLM applications become more personalized, evaluation must account for user-specific factors. Quality definitions may vary based on user preferences, expertise levels, and interaction

history. Context-dependent success criteria mean that the same response may be appropriate in one situation and inappropriate in another.

Future evaluation approaches must address these complexities by incorporating user-specific quality preferences into assessment, developing methods for evaluating long-term relationship quality rather than just single interactions, and ensuring privacy-preserving evaluation methods that do not expose sensitive user data.

14 Limitations

This survey provides practical guidance for LLM evaluation, but it does not solve all challenges. We acknowledge the following limitations in the scope and applicability of this work.

14.1 What This Guide Does Not Cover

This survey focuses on application-level evaluation and does not address several important related topics. Foundation model training evaluation, including the assessment of pre-training dynamics, loss curves, and capability emergence, requires different methodologies than those presented here. Formal verification methods that provide mathematical guarantees about system behavior are not applicable to LLM outputs, which are inherently probabilistic.

We do not provide legal guidance on compliance with AI regulations such as the EU AI Act or sector-specific requirements. Organizations should consult legal experts for regulatory matters. Similarly, we do not deeply address economic trade-offs between evaluation depth and cost, though practitioners must make these trade-offs in practice.

The tooling landscape for LLM evaluation changes rapidly. We mention specific frameworks to provide context and concrete examples, but we do not comprehensively review or recommend specific commercial products.

14.2 Fundamental Challenges That Remain Open

Several fundamental challenges in LLM evaluation remain unresolved and may not have complete solutions. There is no ground truth for subjective dimensions such as helpfulness, tone, and appropriateness. These qualities are inherently in the eye of the beholder, and no evaluation method provides definitive answers about whether a response is "good enough."

Distribution shift remains a persistent challenge. Evaluation on curated test sets cannot guarantee production performance because real users formulate requests in ways that test sets cannot fully anticipate. The gap between offline and online quality is unavoidable.

The adversarial arms race continues to evolve. As safety guardrails improve, adversarial prompts become more sophisticated. Evaluation cannot guarantee safety against novel attacks that have not yet been conceived.

Model inscrutability limits diagnostic capability. We evaluate LLMs effectively as black boxes, and understanding *why* a model fails on specific inputs remains difficult. This limits our ability to predict and prevent failures proactively.

Finally, the rapidly evolving landscape means that best practices documented here may be superseded as the field advances. Readers should stay current with recent literature and adapt these recommendations as new methods emerge.

14.3 Limitations of Specific Methods

Each evaluation method described in this survey has inherent limitations. LLM-as-judge approaches exhibit known biases and have limited domain expertise, making them unsuitable as sole arbiters of quality. Human evaluation is expensive, slow, and subject to annotator fatigue and inconsistency. Automated metrics may not correlate with true quality and can be gamed

through optimization that exploits metric weaknesses. Data contamination becomes increasingly difficult to avoid as training corpora expand to encompass most of the public internet.

14.4 Scope of Applicability

This survey is most applicable to text-in, text-out LLM applications that operate as single-model systems rather than complex multi-agent orchestrations. The methods are best validated for English-language applications and commercial or enterprise deployments with defined quality requirements.

Different evaluation strategies may be needed for multi-modal systems involving vision or audio, complex multi-agent architectures where multiple LLMs coordinate, applications in low-resource languages where evaluation methods are less validated, and creative applications without well-defined notions of correctness.

We encourage researchers and practitioners working in these areas to adapt the principles presented here while developing specialized methodologies appropriate to their contexts.

15 Conclusion

LLM applications require evaluation-driven iteration, not intuition-based prompt engineering. This paper provides a framework and reproducible harness demonstrating that generic prompt improvements are not monotonic—adding "helpful assistant" wrappers with explicit rules degraded extraction by 10% and RAG compliance by 13% while improving instruction-following by 13%. A four-condition ablation isolated the mechanism: the system wrapper itself is benign; the degradation comes from generic rules conflicting with task-specific constraints.

The practical implication is that every prompt change should be validated against task-specific test suites before deployment. The evaluation harness, datasets, and experimental results in this paper provide a starting point for teams to build their own evaluation practices.

Acknowledgments

We thank the broader research community for developing the foundational methods and benchmarks discussed in this survey.

AI Assistance Disclosure. This paper was developed with AI assistance for drafting and organizing content. The human author directed the outline, verified factual claims, curated references, and edited the final text. All citations were verified to correspond to real, published works.

A Appendix

A.1 Full Compatibility Checklist

- **Data Privacy**: Ensure PII is stripped from logs before sending to cloud judges.
- **Token Limits**: Check that RAG prompts + retrieved context fit within the model's effective context window (often shorter than the theoretical max).
- **Rate Limiting**: Implement exponential backoff for API calls to avoid 429 errors during batched evaluation.
- **Versioning**: Hash prompt sets and data splits (e.g., `v1.0-sha256`) to ensure regression tests are comparing apples to apples.
- **Fallbacks**: Ensure the system fails gracefully (e.g., returns a default error message) if the LLM times out or returns malformed JSON.

A.2 Human Evaluation Rubrics

Helpfulness (1-5 Likert)

- **1 (Not Helpful)**: Irrelevant, refuses answer incorrectly, or hallmarks hallucinations.
- **3 (Somewhat Helpful)**: Addresses part of the query but misses nuance or follows wrong format.
- **5 (Very Helpful)**: Directly answers user intent, follows all formatting constraints, and is concise.

Harmlessness (Binary)

- **Safe**: No hate speech, PII leakage, or dangerous instructions.
- **Unsafe**: Contains any prohibited content.

A.3 Extended Monitoring Metrics

- **Token Usage**: Input/output token counts per request to track cost spikes.
- **Cache Hit Rate**: Percentage of similar queries served from semantic cache.
- **Throttling**: Frequency of hitting provider rate limits.
- **User Feedback**: Ratio of thumbs-up/down per model version.
- **Escalation Rate**: Percentage of sessions where user requests a human agent.

A.4 RAG Evaluation Checklist

1. Separate retrieval and generation evaluation.
2. Measure retrieval Recall@k and Precision@k.
3. Evaluate faithfulness to retrieved documents.
4. Check for correct but unsupported" responses.
5. Verify citation accuracy and coverage.
6. Test out-of-scope queries (information not in knowledge base).
7. Monitor retrieval latency and index freshness.

A.5 LLM-as-Judge Checklist

1. Use a different model than the one being evaluated.
2. Provide explicit rubrics in the evaluation prompt.
3. Request chain-of-thought reasoning before scores.
4. Randomize presentation order for comparisons.
5. Validate scores against human judgments on a sample.
6. Use multiple judge models where feasible.
7. Document known biases in your report.